Lec-14

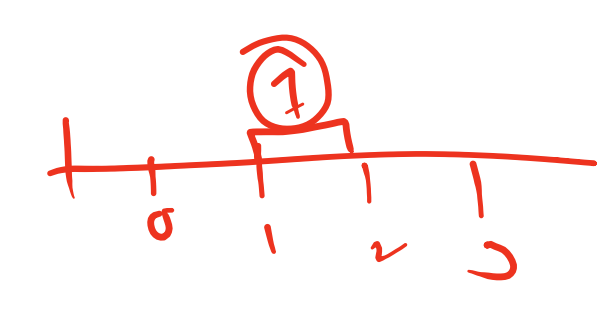
19/10/26 11:08 AM

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `ith` line are `(i, 0)` and `(i, height[i])`.

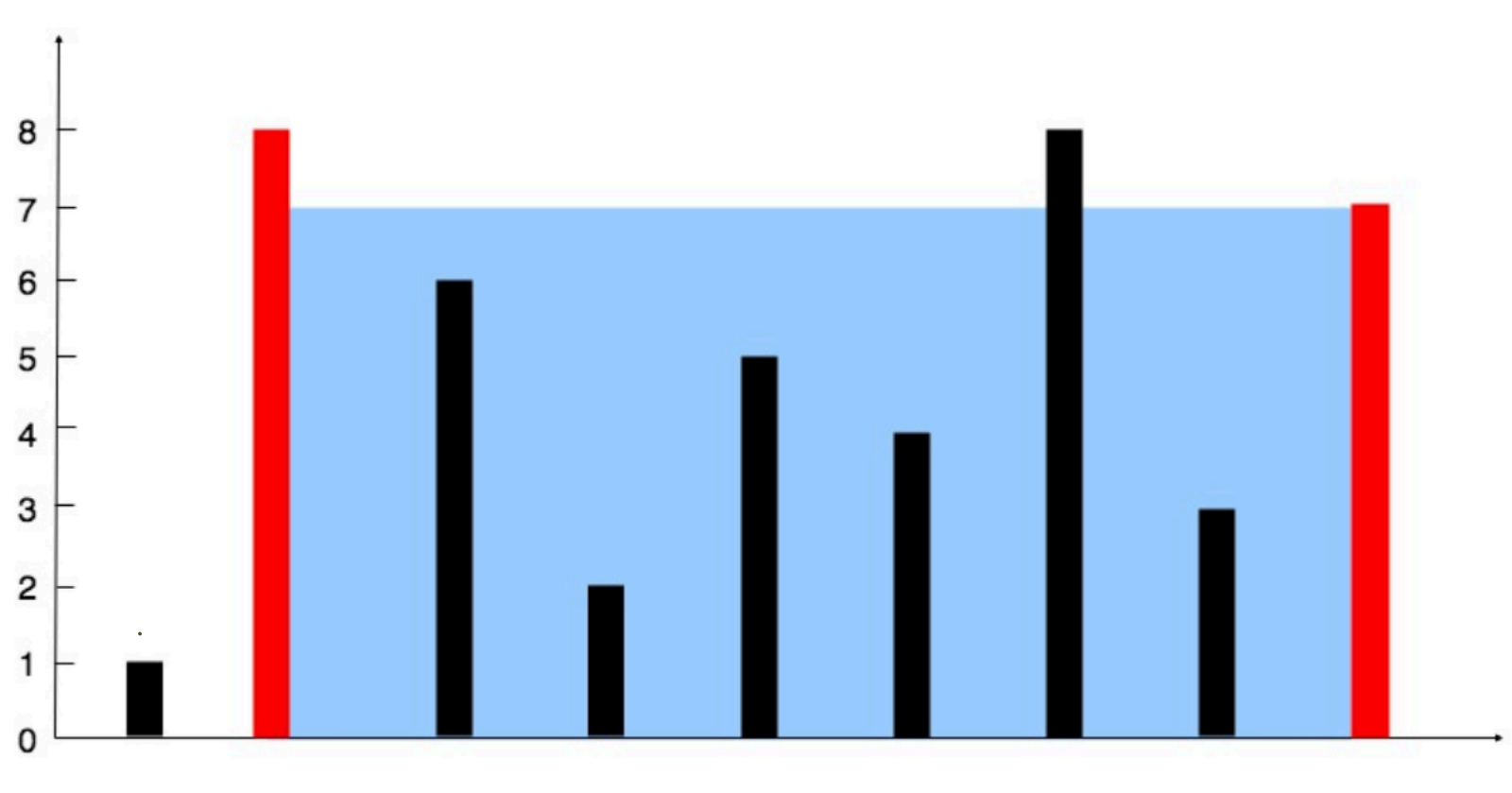
Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

Notice that you may not slant the container.



Example 1:



Two-ptr

Input: height = [1,8,6,2,5,4,8,3,7]

j-i = 8
j-i = 8-1 = 7
= 7-1 = 6
= 6-1 = 5
= 6-2 = 4
= 5-3 = 2

6-4 = 2
6-5 = 1

1 x 8 = 8
7 x 7 = 49 ✓
3 x 6 = 18
8 x 5 = 40
6 x 4 = 24
2 x 2 = 4
5 x 2 = 10

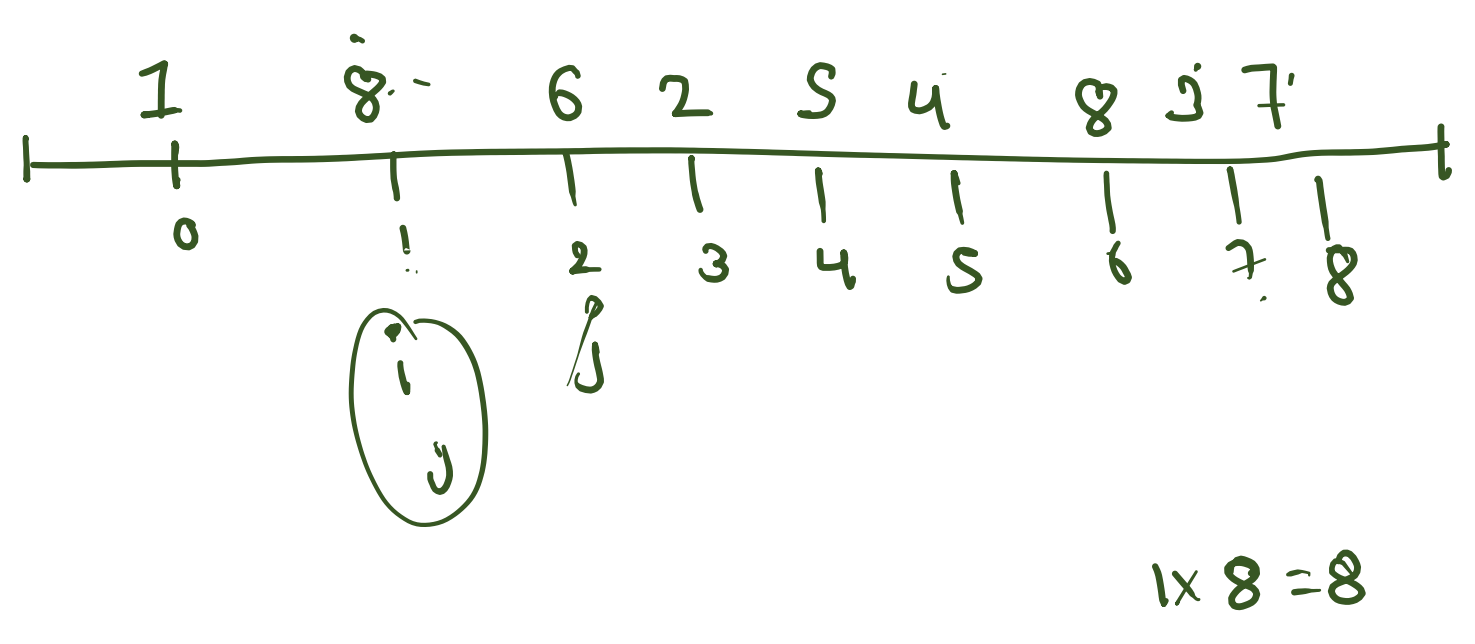
4 x 1 = 4

[1, 8, 6, 2, 5, 4, 8, 3, 7]
0 1 2 3 4 5 6 7 8
j
i

[1, 8, 6, 2, 5, 4, 8, 3, 7]
0 1 2 3 4 5 6 7 8

ans = ~~0~~ ~~8~~ 49

```
while (i < j) {
  int w = (j - i)
  int h = min(arr[i], arr[j])
  ans = max(ans, h x w)
  if (arr[i] < arr[j]) {
     i++
  }
  else {
     j--
  }
}
```



1 x 8 = 8
7 x 7 = 49
3 x 6 = 18
8 x 5 = 40
4 x 4 = 16
2 x 2 = 4
5 x 1 = 5

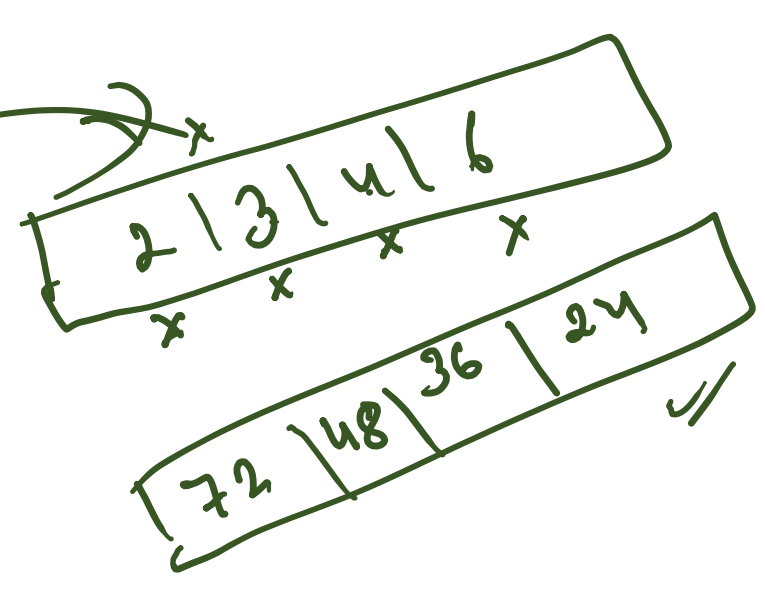
Given an integer array `nums`, return *an array* `answer` *such that* `answer[i]` *is equal to the product of all the elements of* `nums` *except* `nums[i]`.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

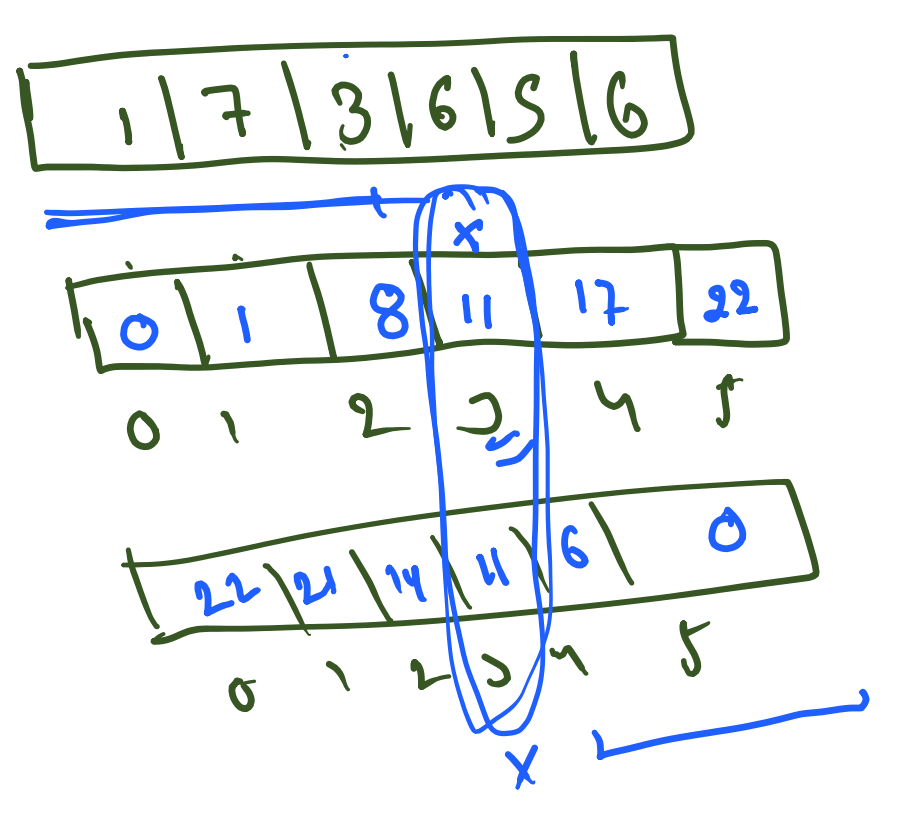
You must write an algorithm that runs in `O(n)` time and without using the division operation.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [24,12,8,6]
```



[1,7,3,6,5,6]
i



915. Partition Array into Disjoint Intervals

Medium · Topics · Companies

Left max ≤ Right min

Given an integer array `nums`, partition it into two (contiguous) subarrays `left` and `right` so that:

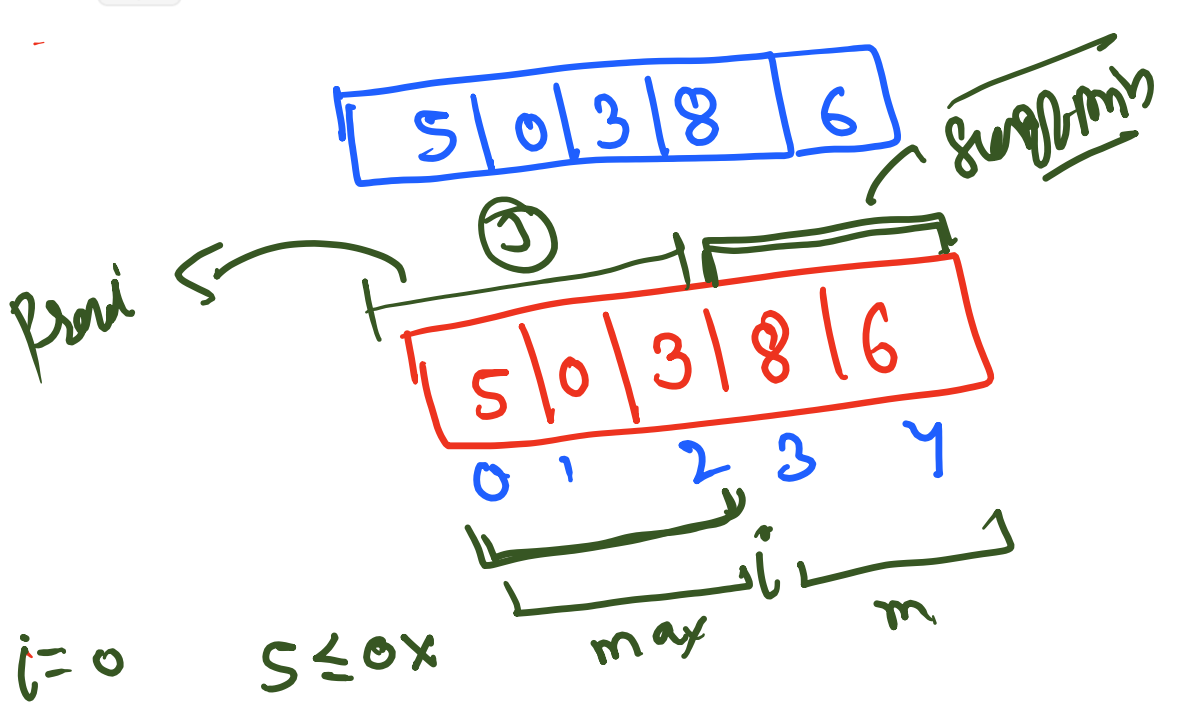
- Every element in `left` is less than or equal to every element in `right`.
- `left` and `right` are non-empty.
- `left` has the smallest possible size.

Return *the length of* `left` *after such a partitioning*.

Test cases are generated such that partitioning exists.

Example 1:

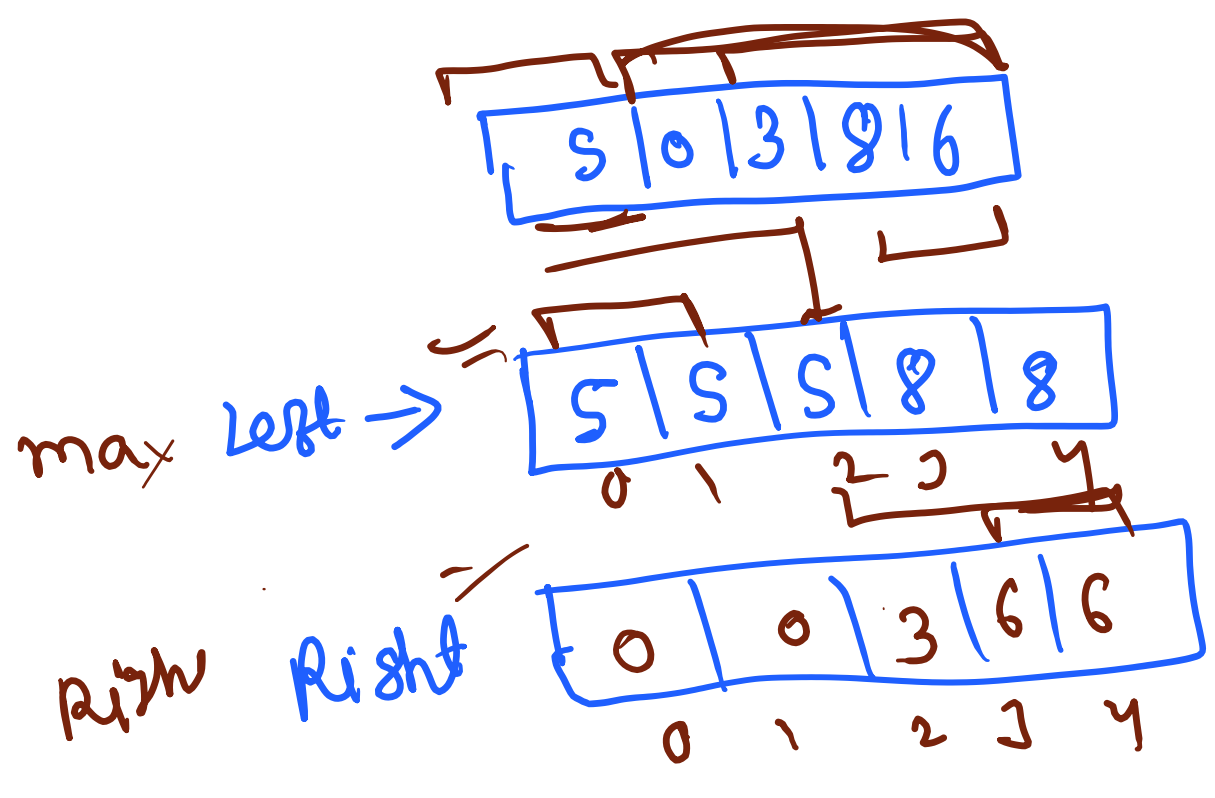
```
Input: nums = [5,0,3,8,6]
Output: 3
Explanation: left = [5,0,3], right = [8,6]
```



i = 0 5 ≤ 0 x
i = 1 5 ≤ 3 x
i = ② 5 ≤ 6 ✓

left[i] ≤ R[i+1]

i+1



i = 0 left(0) ≤ Right(1)
5 ≤ 0 x

i = 1 Left(1) ≤ Right(2)
5 3

i = 2 Left(2) ≤ Right(3)
5 ≤ 6

i+1

169. Majority Element

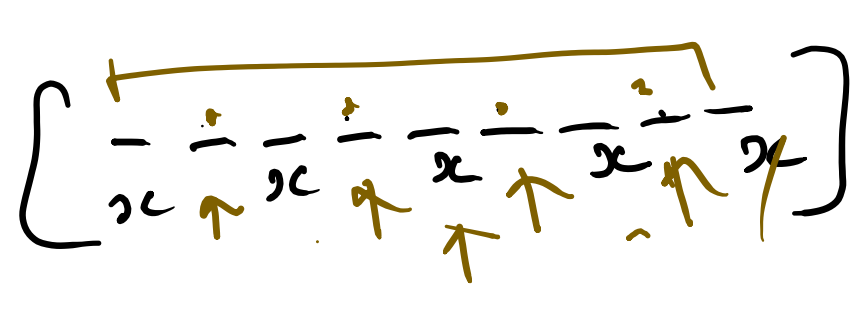
Solved ✓

Easy · Topics · Companies

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

~~7~~ ② 4



{Moore voting}

[0 1 2 3 4 5 6 7 8
2 7 3 7 3 7 7 2 7]

$\frac{9}{2}+1 = 5 \rightarrow$ time

e = ~~2~~ ~~7~~ ~~3~~ ~~7~~ ~~3~~ 7
vote = ~~1~~ ~~0~~
~~1~~ ~~0~~ ~~1~~ ~~0~~
~~1~~ ~~0~~ ~~1~~ ~~1~~ ~~2~~
① ②

```
if (e == arr[i]) {
    vote++
}
else {
    vote--
}
if (vote == 0) {
    e = arr[i]
    vote = 1
}
```

e = x ~~1~~ ~~2~~ ~~3~~ n
v = ~~1~~ ~~0~~ 1 1 1 1 1 1

Binary search

① [2, 3, 5, 7, 9, 11, 13, 14, 15, 18, 20]

```
public static int MooreVoting(int[] arr) {
     int e = arr[0];→2
     int vote = 1;→ 1
     for (int i = 1; i < arr.length; i++) {
       if (e == arr[i]) {
          vote++;
       } else {
          vote--;
       }
       if (vote == 0) {
          e = arr[i];
          vote = 1;
       }
     }
     return e;
   }
```

[0 1 2 3 4 5 6 7 8
2 7 3 7 3 7 7 2 7]

e = ~~2~~ ~~7~~ ~~3~~ ~~7~~ ~~3~~ ⑦

v = ~~1~~ ~~0~~ ~~1~~ ~~0~~ ~~1~~ ~~0~~ ~~1~~ ~~0~~ ~~1~~ ~~2~~ ~~1~~ 2

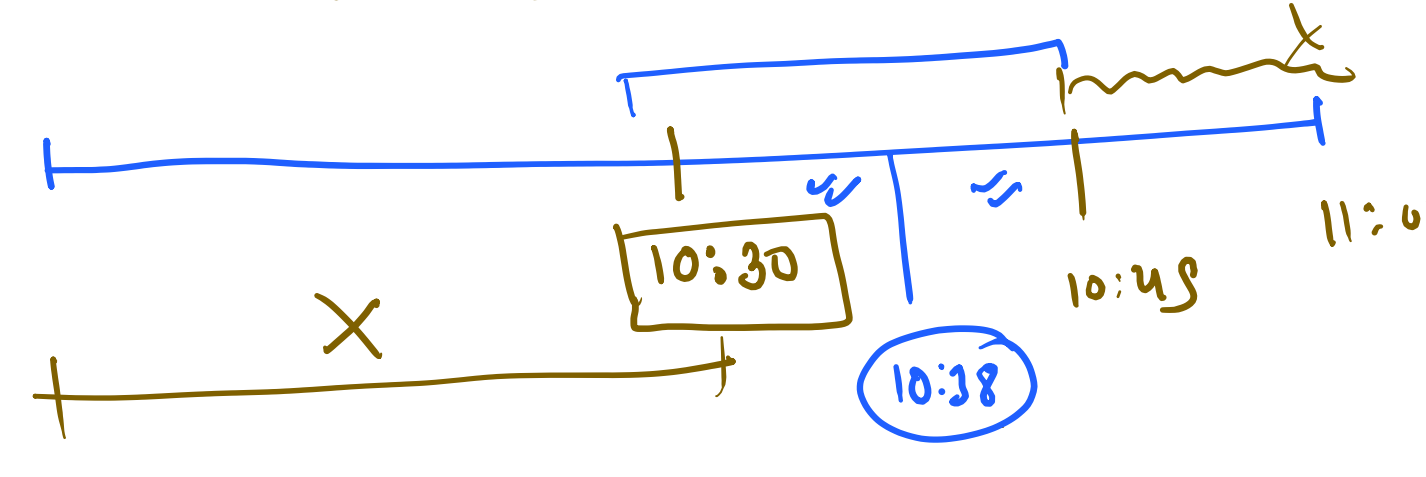
[2, 3, 5, 7, 9, 11, 13, 14, 15, 18, 20]
0 1 2 3 4 5 6 7 8 9 10

only Sorted Array

Sorted Array

Story

10:00 to 11:00



item = 13

item = 12

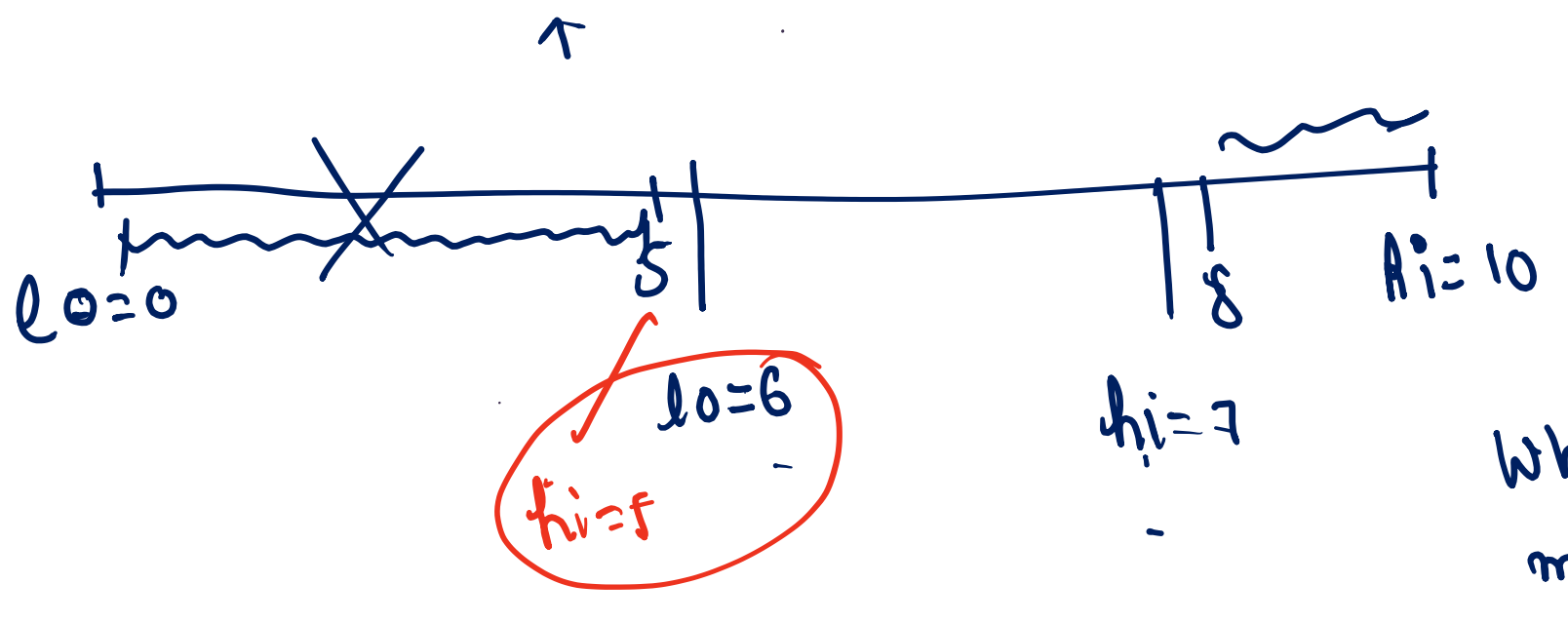
[2, 3, 5, 7, 9, 11, 13, 14, 15, 18, 20]
0 1 2 3 4 5 6 7 8 9 10

11 > 13

15 > 13

15 > 12

13 > 12



mid = $\frac{10+0}{2}$ = 5
= $\frac{10+6}{2}$ = 8
= $\frac{6+7}{2}$ = 6

```
while (lo <= hi) {
  mid = (lo + hi) / 2
  if (arr[mid] == item) {
     return mid
  }
  else if (arr[mid] > item) {
     hi = mid - 1
  }
  else {
     lo = mid + 1
  }
}
```

$x^K \le N$

x → maximum

K = 3

N = 149